

MYSTERY

CASE AUTHORIZING STRESS REPORT

Autorenfähigkeit des implementierten Domain-Modells

READ-ONLY LAB · 3. Oktober 2026

Forge-Dice/Forge · main @ 3d7545d84388

GO WITH CHANGES: Der heutige Domain-Kern ist authorbar, aber noch kein automatischer Produzent lösbarer Fälle.

200 Kandidaten; 128 valide Paare; 528 gezielte Mutationen; 576 NPC-Snapshots; alle acht Semantic-Codes ausgelöst.

Stärkste Grenze: Fakten und Lösung sind formulierbar, der Beweisweg von Evidence zur erforderlichen Conclusion wird nicht validiert.

Minimum: JSON plus kleiner Validator-/Binding-CLI-Wrapper, fünf Templates und zehn Golden Cases. Kein grafischer Editor vor dem Slice.

Keine Änderungen am kanonischen Repository.

Messgrundlage: 128 valide Basisfälle aus 16 Familien, ergänzt um feste Negativvarianten, Mutationen und fokussierte Grenzproben. Ergebnisse gelten für den fixierten Commit und den dokumentierten Versuchsaufbau.

Inhalt

1. Actual Model
2. Case Families
3. Generation Experiment
4. Validation Funnel
5. Authoring Friction
6. Mutation Results
7. False Leads
8. Multi-Actor Cases
9. NPC Knowledge
10. Knowledge Leakage
11. Complexity
12. Canonicalization
13. Five Templates
14. Generation Pipeline
15. Error Feedback
16. Minimum Authoring Tool
17. Ten Golden Cases
18. Model Changes
19. GO/NO-GO Matrix
20. Exact Next Actions
21. Experimental Appendix

Belegarten: VERIFIED FROM CODE · EXPERIMENTALLY VERIFIED · INFERRED · PRODUCT JUDGMENT · UNKNOWN

1. Actual Model

PRODUCT JUDGMENT

Gesamturteil: GO WITH CHANGES für menschlich begleitetes Authoring und experimentelle Build-Time-Generierung. Kein Nachweis für automatisch lösbare, abwechslungsreiche und publikationsreife Mystery-Fälle. Das heutige Modell ist als präziser Fakten- und Antwortschlüssel-Kern brauchbar; seine stärksten Grenzen liegen bei Evidence-Interpretation, Lösbarkeitsnachweis und Autorenführung.

VERIFIED FROM CODE

Untersucht wurde ausschließlich der beim Start per GitHub-GET bestätigte main-Commit 3d7545d843883418348004e68717399a64da7a7d, Tree 67103229baccaefd68ce5047d8503eca9883f1d3. Ein git archive dieses exakten Objekts lieferte den isolierten Quellstand; vorhandene lokale Arbeitsdateien oder frühere Audit-Ergebnisse wurden nicht übernommen. Keine kanonischen Dateien, Branches, Commits, PRs, Settings oder Kommentare wurden verändert.

EXPERIMENTALLY VERIFIED

Im Anhang liegen sieben unveränderte Domain-Quelldateien mit Blob- und SHA-256-Nachweis, der Generator, Rohkandidaten, sämtliche Resultate und Reproduktionshinweise. 452 bestehende Mystery-Tests in sieben Testdateien bestanden. Dies ist kein Forge-Audit; Forge-Code wurde nicht experimentell bewertet.

EXPERIMENTALLY VERIFIED

Messumfang: 200 Kandidaten, davon 128 Basiskandidaten und 72 fest eingeplante Negativvarianten; 528 unabhängige Truth-/Solution-Mutationen; 576 NPC-Snapshots auf 24 Fällen; 120 Multi-Actor-Literalprüfungen; 399 Canonicalization-Prüfreihen; 28 zusätzliche fokussierte Probes. Die Fehlerquote ist durch den Versuchsaufbau vorgegeben und keine Schätzung menschlicher oder LLM-Fehlerraten.

Capability Matrix

VERIFIED FROM CODE

Struktur	CAN REPRESENT	CAN VALIDATE	CAN HASH/BIND	CANNOT REPRESENT / Grenze
CaseTruth	Personen, Orte, Items, Beziehungen, Events, Motive, Propositionen, Evidence, Secrets, RedHerrings	Strenge Shapes, ID-Arten, Eindeutigkeit, Referenzen, Zeiten; danach separate Semantic-Stufe	SHA-256 mit forge-case-c14n-v1; immutable Snapshot	Keine typisierten Todes-, Besitz-, Schadens- oder Zugangs-Zustandswechsel
Semantic Validation	8 FindingCodes; Instant / halboffene Intervalle	Räumliche Widersprüche, drei Claim-Formen, Zeitrichtung und Zyklen	Report trägt rulesetVersion; kein eigener semantisch-validierter Snapshot-Typ	Keine Lösbarkeit, Reisezeiten, Motivplausibilität, Evidence-Wahrheit oder Schuldableitung
CaseSolution	Resolutions, Targets, 3 Rollen, Intent, Mechanism, vollständige/partielle Verantwortung, direkte Ursachen, 6 Conclusion-Formen	Bindung, Referenzen, Duplikate, requiredConclusions gegen true/false/undetermined	caseld + truthHash; eigener Solution-Hash	Kein Evidence→Conclusion-Beweisgraph; keine Accusation-/Verdict-Engine
NPC Knowledge	Awareness, Proposition-Knowledge, Belief, Uncertain, 5 Provenance-Arten	Faktivität von Knowledge, Hashbindung, Referenzen, Erwerbszeiten, strukturelle Claim-Duplikate	truthHash und optional solutionHash; kein eigener NPC-Hash implementiert	Kein Conclusion-Knowledge; keine automatische Deduktion, Sichtlinie oder Glaubwürdigkeitsprüfung
NPC Projection	9 erlaubte Claim-Formen, Stances, lokale Handles	Bindung vor Ausgabe; explizite Konstruktion statt Domain-Objektkopie	Erneute Truth-/Solution-Hashprüfung	Keine Namen, Descriptions, Provenance, Resolutions oder öffentliche Display-Zuordnung

VERIFIED FROM CODE

CaseTruth enthält genau drei objektive Claim-Kinds: personAt, eventHasParticipant und eventHasItem. CaseSolution ergänzt personResponsibleForEvent, personRoleForEvent, noPersonResponsibleForEvent, eventCausedEvent, eventIntent und eventMechanism. NPCs können diese neun Formen subjektiv referenzieren. Ein Feld description, relationship.kind oder motive.description ist Text und liefert keine maschinenprüfbare zusätzliche Semantik.

VERIFIED FROM CODE

DESIGNED ONLY: main enthält einen historischen TASK-0005-Architekturreview und erklärt die Dialogue-Implementierung ausdrücklich als nicht enthalten. Der dort beschriebene Policy-Ansatz ist keine ausführbare Fähigkeit. PlayerRef V1, Accusation/Verdict, Investigation/Evidence Access, Interrogation/Release, CasePackage, PlayerKnowledge, Session/Replay und Save/Load sind im aktuellen main nicht implementiert. Ob dafür außerhalb dieses Snapshots fertige aktuelle Entwürfe existieren, wurde nicht untersucht.

UNKNOWN

UNKNOWN gilt für den aktuellen Design- und Freigabestand dieser außerhalb von main liegenden Komponenten. Frühere Gesprächsstände wurden nicht als Implementierungsbeleg verwendet.

Belegkonvention

Alle Abschnittsaussagen sind mit VERIFIED FROM CODE, EXPERIMENTALLY VERIFIED, INFERRED, PRODUCT JUDGMENT oder UNKNOWN gekennzeichnet. Ein Label über einer Tabelle gilt für deren Datenzeilen. Codebelege C1-C8 und Datendateien sind in Abschnitt 21 eindeutig aufgeschlüsselt; die Statusurteile in Abschnitt 19 sind Produkturteile, keine Testergebnisse.

2. Case Families

EXPERIMENTALLY VERIFIED

Die folgenden 16 Familien wurden je achtmal als strukturierte Kandidaten erzeugt. Familiennamen bezeichnen Authoring-Ziele. Die Generatorfälle sind strukturell verschiedene Testmodelle, keine 128 ausformulierten spielbaren Geschichten. Einige Familien teilen bewusst dieselbe formale Ausdrucksgrenze; ein Label wie murder erzeugt keinen typisierten Todeszustand.

VERIFIED FROM CODE

Familie	Bestehende Strukturen	Fehlende oder nicht geprüfte Semantik
Mord	Events/Teilnehmer/Items; Target Person; Verantwortliche; intended; ordinary	Kein death-Eventtyp, Lebenszustand, Tötungsmechanismus oder Beweis der Täterschaft
Sabotage	Vorbereitungs- und Folgeevent, direkte Kanten; Item-Target; intended	Kein Funktionszustand, Schaden oder Gegenfaktum „ohne Eingriff intakt“
Diebstahl	Items an Events, Personenpräsenz; Item-Target; Verantwortung	Besitz, Eigentum, Entnahme und Übergabe nicht formal ausdrückbar
Verschwundene Person	Letzte belegte Präsenz, einzelne positive/negative personAt-Claims	Keine Quantifikation „seitdem nirgends gesehen“, kein Aufenthaltsstatus unbekannt
Inszeniertes Verschwinden	Person als Planer/Actor; intended; falsche Präsenzproposition, NPC-Belief	Inszenierung als Handlungstyp und öffentliche Scheinlage fehlen
Unfall	complete mit leeren assignments; unintended; ordinary; Ursachen	Naturkausalität und Fahrlässigkeit werden nicht abgeleitet; Unfall kann auch menschliche Verantwortung haben
Zwei Verantwortliche	2 assignments; Rollen; einzelne Conclusions je Person	Kein automatisches „genau diese beiden“ als Spielerantwort oder Negativschluss
Drei Verantwortliche	3 assignments; mehrere Rollen/Conclusions	Kooperationsplan, individuelle Absichten und gemeinsames Delikt nicht eigenständig modelliert
Auftraggeber + Ausführer	planner + direct_actor; getrennte Vorbereitungs-/Ausführungsevents	Auftrag selbst nur Ereignistext; keine normative Hierarchie zwischen Personen
Helfer + Ausführer	facilitator + direct_actor; räumlich getrennte Ereignisse	Beihilfegrad, Wissen um den Zweck und individuelle Schuldgrade fehlen
Falsches Alibi	personAt(...)=false, Evidence supports, NPC belief:true	Keine Lügenabsicht in NPC-Knowledge und keine ausgeführte Dialogue Policy
Übernatürliche Ursache	mechanism supernatural/mixed, Ursachen, ggf. keine verantwortliche Person	Kein Spirit-/Entity-Typ oder Magieregelsystem; Bilokation desselben Person-IDs bleibt Konflikt
Technischer Defekt	Item-Ziel; noPersonResponsible; ordinary; Ereigniskette	ordinary bedeutet nicht speziell technisch; kein Zustandsautomat des Bauteils

Familie	Bestehende Strukturen	Fehlende oder nicht geprüfte Semantik
Mehrere Ursachen	Mehrere causedByEventIds, causesComplete, edge-Conclusions	Keine AND/OR-/notwendige-/hinreichende-Ursache; keine transitive Conclusion
Partielle Verantwortung	partial; bekannte assignments; unknown externe Personen	„Unbekannter weiterer Täter“ kein eigener Person-Platzhalter; Nichtaufzählung ist kein Ausschluss
Eine Person, mehrere Rollen	Eine assignment mit planner + direct_actor	Absicht gilt pro Event, nicht individuell pro Rolle/Person

PRODUCT JUDGMENT

Die vorhandene Struktur deckt räumliche Indizien, direkte Kausalität und Antwortschlüssel breit ab. Für Besitzrätsel, belastbare forensische Schlussketten oder polymorphe übernatürliche Regeln reicht sie allein nicht. Zusätzliche Felder sollten erst aus einem konkreten spielbaren Fall folgen; neutrale Events dürfen als Abstraktion dienen, aber Text darf fehlende Prüfsemantik nicht vortäuschen.

3. Generation Experiment

EXPERIMENTALLY VERIFIED

Ein deterministischer Scratch-Generator erzeugte Seeds 1000-1127 mit genau acht Beispielen pro Familie. Die Ausgangsgrößen decken jede Anzahl von 3 bis 8 Personen, 2 bis 6 Orten, 3 bis 12 Events und 1 bis 5 Items ab. Er variiert Instant-/Interval-Zeiten, Teilnehmer-/Item-Listen, Evidence-Quellen und Richtungen, drei Proposition-Kinds, Beziehungen, Motive, Secrets, RedHerrings, Kausalitätsmuster sowie Solution-Rollen, Intent, Mechanism und Completeness. Motive, Secrets und RedHerrings kommen mit jeweils 1-3 Einträgen vor; Evidence hat 1-2 Links.

EXPERIMENTALLY VERIFIED

128 verschiedene Truth-Hashes wurden gemessen. Auch nach Entfernung von Seed, Familienlabel, caseld, Titel, Namen, Beschreibungen, Revision und truthHash bleiben 128 verschiedene strukturierte Truth-/Solution-Paare. Die Vielfalt kommt somit nicht ausschließlich aus anderen Bezeichnern. Sie beweist allerdings keine narrative Vielfalt.

EXPERIMENTALLY VERIFIED

Der Generator baut bewusst konsistente Grundgerüste mit überwiegend zeitlich getrennten Events. CaseTruth wird beim Berechnen des Solution-Bindings geparst; danach durchläuft jeder vollständige Kandidat erneut den echten Funnel. Es gibt keine Schleife „erzeugen, bis grün“. 72 Negativvarianten aus zwölf festen Fehleroperatoren wurden vor dem Funnel hinzugefügt und nicht repariert oder verworfen.

UNKNOWN

Methodische Grenze: keine echte Mensch-Autorenstudie und keine Claude-/Codex-Generierung. 128 grüne Basiskandidaten sagen etwas über Ausdruck und konsistente Konstruktion aus, nicht über eine Erstversuchsquote eines LLM. Zeitliche Überschneidungen und Randbedingungen sind im Mutationsteil und in fokussierten Probes abgedeckt.

EXPERIMENTALLY VERIFIED

Während der Instrumentierung wurde eine Seed-Korrelation in der Größenverteilung erkannt. Vor dem finalen Lauf wurden Seed-Mischung und Personenzahl-Stratifizierung korrigiert, um alle geforderten Größen zu erreichen. Die Negativoperatoren blieben unverändert. Ausschließlich der finale Lauf liegt den Zahlen im Bericht zugrunde.

EXPERIMENTALLY VERIFIED

Zusätzliche erhaltene Überlappungsserie: 128 Kandidaten, 122 PASS und 6 Semantic-Rejects. Bei Seeds 1015, 1025, 1060, 1070, 1095 und 1120 wurde ein Event räumlich/zeitlich an ein anderes angeglichen. Die ältere Alibi-Ableitung über Ortsindex produzierte dabei einen angeblich falschen personAt-Claim am tatsächlichen Ort. CLAIM_TRUTH_CONFLICT und PERSON_PRESENCE_NEGATED deckten die Generator-Annahme auf. Diese sechs Fehlschläge wurden weder repariert noch aus dem Appendix entfernt; die Serie wird separat vom festen 200er-Funnel gezählt.

4. Validation Funnel

EXPERIMENTALLY VERIFIED

Stufe	Input	Abgelehnt	Weiter / Ergebnis
Raw Candidate	200	0 vorab aussortiert	Alle 200 geprüft
CaseTruth parser	200	24	176 schema- und referenzgültig
Semantic Validator	176	24	152 ohne Findings
CaseSolution parser/binding	152	24	128 gebundene valide Paare

EXPERIMENTALLY VERIFIED

Fehlergruppe	Fälle	Tatsächliche Reaktion
Missing reference, duplicate ID, wrong ID kind, invalid interval	je 6	CaseTruth parser
Claim mismatch, reversed cause, Person-/Item-Doppelort	je 6	Semantic Validator
Stale truthHash	6	Binding-Mismatch in Solution Parser
Unmögliches Required-Literal	6	Resolved-Wert widerspricht Literal
Doppelter Conclusion-Claim	6	Same claim as existing conclusion
Unbekannte requiredConclusion	6	Unknown conclusion

EXPERIMENTALLY VERIFIED

Nur sechs der 24 Solution-Fehler sind tatsächliche Hash-Binding-Fehler. Die anderen 18 betreffen interne Solution-Referenzen oder die Antwortschlüssel-Konsistenz. Der Bericht trennt diese Kategorien, obwohl sie denselben öffentlichen Parser verwenden.

EXPERIMENTALLY VERIFIED

FindingCode	Funnel-Vorkommen	Mutations-Vorkommen
PARTICIPANT_CLAIM_MISMATCH	6	48
CAUSAL_CYCLE	6	24
CAUSE_STARTS_AFTER_EFFECT	6	24
PERSON_LOCATION_CONFLICT	12	48

EXPERIMENTALLY VERIFIED

FindingCode	Funnel-Vorkommen	Mutations-Vorkommen
PERSON_PRESENCE_NEGATED	6	48
ITEM_LOCATION_CONFLICT	6	24
CLAIM_TRUTH_CONFLICT	0	24
ITEM_CLAIM_MISMATCH	0	24

EXPERIMENTALLY VERIFIED

Mehrere Findings pro Kandidat sind möglich. Beispiel: Die nachträglich eingefügte Rückwärtskante $e1 \rightarrow e0$ erzeugt sowohl einen Zeitrichtungsfehler als auch den Zyklus $e0 \leftrightarrow e1$. Eine separate azyklische Rückwärtskante isoliert CAUSE_STARTS_AFTER_EFFECT; ein Zyklus bei gleichen Startzeiten isoliert CAUSAL_CYCLE.

INFERRED

„Häufig“ bedeutet hier die Frequenz innerhalb eines bewusst ausgewogenen Testkorpus. Aus zwölf gleich oft injizierten Fehlern lässt sich kein Ranking realer Autorenfehler ableiten. Erwartbar arbeitsintensiv sind Referenzpflege, redundante Präsenzclaims, die Required-Literal-Auswahl und Hashneubindung nach Inhaltsänderungen.

5. Authoring Friction

PRODUCT JUDGMENT

Bereich	Einordnung	Beobachtung und kleinste Entlastung
IDs und Referenzen	GOOD FRICTION + TOOLING GAP	Typpräfix und Existenz verhindern Fehler. Autor-Symbole/ID-Katalog sowie lokal auflösbare Pfade vermeiden Tippfehler.
Zeit und Präsenz	GOOD FRICTION + DOCUMENTATION GAP	Halboffene Intervalle und Teilnehmer→Präsenz sind korrekt, aber leicht falsch verstanden. Timeline-Ansicht als CLI-Ausgabe genügt zunächst.
causedByEventIds	GOOD FRICTION + DOCUMENTATION GAP	Feld steht am Effekt und listet unmittelbare Ursachen. Keine transitive Verantwortungsübertragung; graphische Skizze im Autorenleitfaden.
Redundante Propositionen	BAD FRICTION + TOOLING GAP	Event-Liste und deklarierter truth-Wert müssen übereinstimmen. Autorentooling kann Claims vorschlagen und Widersprüche auf gemeinsame Quelle zurückführen.
Evidence links	MODEL GAP für Beweisführung	supports/refutes verweist nur auf Propositionen und beweist weder Wahrheit noch Verantwortung. Kein automatisches Umdrehen der Richtung.
RedHerrings	DOCUMENTATION GAP + MODEL GAP	Wahrheit des Ziels und Verbindung der Evidence zum Ziel werden nicht geprüft; falsche Conclusions sind nicht direkt adressierbar.
Responsibility	GOOD FRICTION + DOCUMENTATION GAP	Verantwortung ist unabhängig von Anwesenheit. Planer dürfen abwesend sein. Keine automatische Ableitung aus participantIds.
requiredConclusions	BAD FRICTION + TOOLING GAP	Semantik und Referenzen sind wiederholt zu pflegen; Auflistung aller Conclusion-Statuswerte fehlt als öffentliche Diagnose.
partial / complete / null	GOOD FRICTION + DOCUMENTATION GAP	Nichtwissen wird geschützt. partial ist keine Schreibabkürzung für complete; roles:null ist kein leeres Rollenset.
NPC knowledge	GOOD FRICTION + TOOLING GAP	Wissen muss faktiv sein; Alias-Claims dürfen nicht doppelt vorkommen. Ein Snapshot-Assistent kann Binding und Stance-Tabelle erzeugen.
Provenance	DOCUMENTATION GAP	Annotation mit Zeitprüfung, kein Wahrnehmungsbeweis. witnessed verlangt keine physische Event-Teilnahme.
Hashpflege	BAD FRICTION bei Handarbeit; fachlich GOOD	Jede Text-/Revisionsänderung bindet neu. Hashes deterministisch berechnen, niemals vom LLM erfinden lassen.
Namenlose Projektion	TOOLING GAP außerhalb Kernmodell	Sicherer Kontext enthält lokale Handles. Ein späterer freigegebener Darstellungsadapter muss verständliche Bezeichnungen liefern.

PRODUCT JUDGMENT

Keine dieser Einordnungen rechtfertigt automatisch ein neues Domain-Feld. Insbesondere IDs, Hashes, Pfade, Schlusswert-Vorschauen und zeitliche Übersicht sind zunächst Werkzeug- oder Dokumentationsaufgaben.

6. Mutation Results

EXPERIMENTALLY VERIFIED

24 unveränderte gültige Ausgangsfälle wurden jeweils mit 22 unabhängigen Operatoren mutiert: 528 Truth-/Solution-Mutationen. Jeder Lauf startete von einer frischen Kopie, nicht vom vorherigen Mutanten. Die Mutation betrifft eine logische Änderung; zum Beispiel fügt der Doppelort-Operator ein einzelnes kollidierendes Event hinzu. Es wurden keine Produktionsfunktionen verändert.

EXPERIMENTALLY VERIFIED

Mutation	Erkannt	Erste Schicht	Reaktion
missing_reference	24 / 24	Schema	Unresolved reference "person:absent"
duplicate_id	24 / 24	Schema	Duplicate ID "person:p0"
wrong_id_kind	24 / 24	Schema	Expected an ID of the form "location:<slug>"
invalid_interval	24 / 24	Schema	Interval requires start < end
claim_mismatch	24 / 24	Semantic Validator	PARTICIPANT_CLAIM_MISMATCH
reversed_cause	24 / 24	Semantic Validator	CAUSAL_CYCLE, CAUSE_STARTS_AFTER_EFFECT
person_two_places	24 / 24	Semantic Validator	PERSON_LOCATION_CONFLICT, PERSON_PRESENCE_NEGATED
item_two_places	24 / 24	Semantic Validator	ITEM_LOCATION_CONFLICT
contradictory_propositions	24 / 24	Semantic Validator	CLAIM_TRUTH_CONFLICT, PARTICIPANT_CLAIM_MISMATCH
item_claim_mismatch	24 / 24	Semantic Validator	ITEM_CLAIM_MISMATCH
presence_negated	24 / 24	Semantic Validator	PERSON_PRESENCE_NEGATED
self_cause	24 / 24	Schema	An event must not list itself as its cause
stale_truth_hash	24 / 24	Solution Parser	truthHash does not match the given CaseTruth snapshot
impossible_required	24 / 24	Solution Parser	"conclusion:c0" is required to be false but resolves to true
duplicate_conclusion_claim	24 / 24	Solution Parser	Same claim as "conclusion:c0"
unresolved_required	24 / 24	Solution Parser	Unknown conclusion "conclusion:missing"
responsibility_removed	24 / 24	Solution Parser	"conclusion:c0" is required to be true but resolves to false
roles_null_required	24 / 24	Solution Parser	"conclusion:c1" is required to be true but resolves to undetermined
complete_to_partial	24 / 24	Solution Parser	"conclusion:c5" is required to be false but resolves to undetermined
intent_null_required	24 / 24	Solution Parser	"conclusion:c6" is required to be true but resolves to undetermined

EXPERIMENTALLY VERIFIED

Mutation	Erkannt	Erste Schicht	Reaktion
mechanism_null_required	24 / 24	Solution Parser	"conclusion:c7" is required to be true but resolves to undetermined
unknown_responsible_person	24 / 24	Solution Parser	Unknown person "person:missing"

EXPERIMENTALLY VERIFIED

Zusätzlich wurden false knowledge, stale truth hash und stale solution hash jeweils auf 24 NPC-Snapshots angewendet und in allen 72 Fällen vom NPC Parser abgelehnt. Diese Tests gehören zur separaten NPC-Serie und sind nicht in den 528 enthalten.

EXPERIMENTALLY VERIFIED

Kein untersuchter ungültiger Mutant wurde entgegen dem tatsächlich implementierten Schichtenvertrag von der falschen Schicht behandelt. Auffällig, aber beabsichtigt: Selbstkausalität scheitert schon am Event-Schema, ein mehrgliedriger Zyklus erst an Semantic Validation. Referenzintegrität gehört hier zum CaseTruth parser, nicht zur separaten Semantic-Stufe.

EXPERIMENTALLY VERIFIED

roles:null und partial sind für sich genommen zulässig. Sie scheitern in der Mutationstabelle nur, weil bestehende Required-Literale damit undetermined werden. Fokussierte Positivkontrollen mit Verantwortung ohne Rollenpflicht und mit ausschließlich bekannten positiven Aussagen passieren. Ein Test, der null oder partial generell als Defekt zählt, wäre falsch.

EXPERIMENTALLY VERIFIED

Die wichtige Schichtenlücke liegt im Aufrufpfad: parseCaseSolution akzeptiert eine korrekt neu gebundene Truth mit Kausalzyklus, wenn der Aufrufer Semantic Validation überspringt. Das ist gemäß Parservertrag kein Defekt; als Autoren-Importweg ohne vorgeschaltetes Gate wäre es jedoch unzureichend.

7. False Leads

EXPERIMENTALLY VERIFIED

Szenario	Echtes Ergebnis	Präzise Bedeutung
Falsche objektive Proposition	PASS	Eine wohlgeformte Behauptung mit truth:false ist zulässig; „objektive Proposition“ heißt nicht „wahre Behauptung“.
Wahre Köderinformation als RedHerring-Ziel	PASS	misleadingPropositionId muss derzeit nicht auf truth:false zeigen.
Verantwortungsclaim in CaseTruth-Proposition	Schema-Reject	personResponsibleForEvent ist kein ClaimSchema-Kind.
conclusion:-ID in misleadingPropositionId	Schema-Reject	Das alte Modell erlaubt nur proposition:-IDs.
Falsches Alibi über personAt	PASS	Ein räumlicher Claim kann falsch sein; NPC kann ihn glauben und Evidence kann ihn scheinbar stützen.
Evidence supports falsche Proposition	PASS	supports ist keine Garantie der objektiven Wahrheit.
Evidence refutes wahre Proposition	PASS	Auch die umgekehrte Richtung wird nicht auf Wahrheit geprüft.
RedHerring-Evidence ohne Link zum Ziel	PASS	Der Parser prüft IDs, nicht die inhaltliche Verbindung.
Wahrer Köder + falscher Täterglaube im NPC	PASS	Beide Aussagen sind separat repräsentierbar. Ein formaler Inferenzauslöser zwischen ihnen fehlt.

VERIFIED FROM CODE

Ein wahrer Fingerabdruck kann als Evidence-Text beschrieben werden; dessen zeitliche Herkunft lässt sich über passende Events und eventHasItem-/participant-Claims konkretisieren. „Fingerabdruck von X“ selbst besitzt jedoch keinen eigenen Claim-Typ. Die Formulierung darf deshalb nicht als forensisch verifiziert gelten. Gleiches gilt für „Tatwaffe“: eventHasItem bedeutet Anwesenheit eines Items im Event, nicht dessen instrumentelle Verwendung.

EXPERIMENTALLY VERIFIED

Falsche Verantwortungsfolgerungen können als nicht erforderliche Conclusion angelegt und als NPC belief:true repräsentiert werden, obwohl die Resolution diese Person bei complete ausschließt. Das belegt subjektive Misdirection. Es belegt keinen Evidence→Conclusion-Link und keinen spielerseitigen Interpretationszustand.

PRODUCT JUDGMENT

Conclusion-basierte Misdirection wird benötigt, sobald Autoren explizit sagen müssen: Diese wahre Präsenzinformation verführt zu genau dieser falschen Täter-/Rollen-/Ursachenfolgerung; jene spätere Information korrigiert sie. Dafür ist künftig ein eigener Interpretations-/Begründungszusammenhang sinnvoll. Er muss erst an einem konkreten Fall spezifiziert werden.

PRODUCT JUDGMENT

Nicht in das alte RedHerring-Modell zwingen: Schuld als Anwesenheit umkodieren; conclusion:-IDs als proposition:-Strings tarnen; truth absichtlich falsch setzen, um Täuschung zu simulieren; Einträge zu Wahrheit oder Evidence-Richtung automatisch „reparieren“. Das zerstört die Trennung von objektiver Welt, subjektivem Glauben und Schlussfolgerung.

PRODUCT JUDGMENT

Für einen ersten Slice genügt ein bewusst begrenzter falscher Präsenzhinweis plus menschlich geprüfte Interpretation. Für automatisierte komplexe falsche Fährten lautet die Bewertung NO-GO, solange weder formale Interpretation noch Lösbarkeitsprüfung vorhanden sind.

8. Multi-Actor Cases

EXPERIMENTALLY VERIFIED

Zehn Verantwortungs-Konfigurationen wurden mit sechs Claim-Fragen und jeweils beiden Boolean-Literalen geprüft: 120 Versuche, 50 akzeptiert und 70 abgelehnt. Für einen determinierten Claim ist genau ein Boolean zulässig; bei undetermined scheitern beide. Die Zahl 70 ist deshalb keine Fehlerquote.

EXPERIMENTALLY VERIFIED

Konfiguration	p0 zust.	p4 zust.	niemand	p0 actor	p0 planner	p0 helper
one	T	F	F	T	F	F
two	T	F	F	T	F	F
three	T	F	F	T	F	F
planner_actor	T	F	F	F	T	F
facilitator_actor	T	F	F	F	F	T
multi_role	T	F	F	T	T	F
none	F	F	T	F	F	F
partial_empty	U	U	U	U	U	U
partial_one	T	U	F	T	F	F
null_role	T	F	F	U	U	U

EXPERIMENTALLY VERIFIED

T = true, F = false, U = undetermined. p4 ist die nicht zugewiesene Vergleichsperson. multi_role enthält planner + direct_actor für p0. null_role enthält eine bekannte Verantwortung mit unbestimmter Rolle.

VERIFIED FROM CODE

Die Dateneingabe ist einfach: eine assignment pro Person; mehrere Rollen in deren roles-Liste. Dieselbe Person in zwei assignments ist ein Duplikat, nicht die korrekte Mehrrollen-Darstellung. complete mit assignments:[] bedeutet niemand verantwortlich; partial mit assignments:[] bedeutet Verantwortung unbestimmt. Bekannte Rollenlisten sind auch unter partial geschlossen.

EXPERIMENTALLY VERIFIED

requiredConclusions werden mühsam, sobald „genau diese Täter“, einzelne Rollen und Ausschlüsse verlangt werden. complete schließt nicht aufgeführte Personen beim Auflösen eines Claims aus, erzeugt aber selbst keine zusätzlichen requiredConclusions. Die Positivkontrolle mit nur einer erforderlichen positiven Conclusion wird akzeptiert, obwohl weitere Verantwortliche oder Ausschlüsse im Modell existieren.

INFERRED

Shotgun-Frage: Eine spätere Verdict-Engine könnte durch ein bloßes Subset-Matching „beschuldige alle“ fälschlich akzeptieren. Im aktuellen main existiert diese Engine nicht; eine ausnutzbare Shotgun-Lücke wurde daher nicht live nachgewiesen. Wohl nachgewiesen: Der Solution Parser erzwingt kein vollständiges Antwortformat und akzeptiert auch ein bewusst autorisiertes „alle Personen verantwortlich“.

PRODUCT JUDGMENT

Empfehlung: Für den Slice zuerst festlegen, ob die Antwort exakte Personen-/Rollenmengen oder explizite Literale verlangt. Danach Golden Cases für fehlenden Mittäter, zusätzlichen Unschuldigen und offene partial-Verantwortung prüfen. Nicht aus Bequemlichkeit partial→complete ändern, um ein undetermined verschwinden zu lassen.

9. NPC Knowledge

EXPERIMENTALLY VERIFIED

24 gültige Fälle erhielten je 24 Snapshot-Varianten, insgesamt 576. 312 Snapshots wurden akzeptiert, 264 abgelehnt. Alle akzeptierten Snapshots wurden mit projectNpcKnowledge projiziert. Pro Variante existieren 24 unabhängige Ergebnisse.

EXPERIMENTALLY VERIFIED

Varianten	Ergebnis	Bedeutung
knowledge true; knowledge false	48 PASS	Wahre Behauptung kennen bzw. Verneinung einer falschen Behauptung kennen.
false belief	24 PASS	belief darf vom objektiven truth-Wert abweichen.
uncertain true / false / null	72 PASS	Tendenz ist kein Knowledge und darf irren oder offen sein.
Evidence-/Event-Awareness	48 PASS	Kenntnis des Existierens erweitert keine Claims automatisch.
witnessed_event als Nichtteilnehmer	24 PASS	Zeit und Event-Referenz stimmen; tatsächliche Wahrnehmung wird nicht bewiesen.
told_by_person	24 PASS	Existenz des Informanten wird geprüft, nicht dessen Wissen oder tatsächliche Kommunikation.
prior knowledge zu späterem Event	24 PASS	Prior-Label mit acquiredAt:0 ist formal zulässig; Plausibilität ist Autorensache.
author_modeled_inference	24 PASS	Conclusion-Belief möglich; keine maschinelle Ableitungsprüfung.
widersprüchliche räumliche Beliefs	24 PASS	Zwei verschiedene Claims dürfen subjektiv unvereinbar sein.
false knowledge; stale truthHash; stale solutionHash	72 REJECT	Faktivität bzw. Binding verletzt.
Conclusion knowledge	24 REJECT	V1 verbietet diesen Stance-Typ an Conclusions.
witnessed falscher Zeitpunkt; prior nonzero; acquiredAt > asOf	72 REJECT	Erwerbszeitregeln verletzt.

EXPERIMENTALLY VERIFIED

Varianten	Ergebnis	Bedeutung
duplicate attitude; unknown provenance; unknown awareness; duplicate awareness	96 REJECT	Referenz-/Eindeutigkeitsregeln greifen.

EXPERIMENTALLY VERIFIED

Ein zusätzliche Probe mit zwei verschiedenen Proposition-IDs für denselben Claim wird als doppelte Attitude abgelehnt. Der Parser gruppiert nach subjektivem Namespace plus den strukturellen Claim-Feldern, nicht bloß nach Subject-ID. Falsche und widersprüchliche Beliefs zu verschiedenen Claims sind absichtlich erlaubt; zwei Stances zum selben Claim im selben Snapshot nicht.

VERIFIED FROM CODE

Autoren müssen Subject, Stance, acquiredAt, Provenance, asOf und bis zu zwei Hashes konsistent halten. Knowledge über eine eigene verantwortliche Rolle bleibt sogar für den Täter verboten; ein solcher innerer Zustand wird in V1 als Belief geführt. Das ist eine explizite Beschränkung, kein bei diesem Audit gefundenes Parserversehen.

PRODUCT JUDGMENT

Der Aufwand ist mit einer tabellarischen Snapshot-Vorlage und automatischem Binding beherrschbar. Ein Werkzeug sollte Warnungen für unwahrscheinliches Vorwissen und un plausible Witness-Provenance ausgeben können; harte physische Anwesenheitspflicht wäre fachlich zu stark, weil Beobachtung aus Distanz möglich sein kann.

10. Knowledge Leakage

EXPERIMENTALLY VERIFIED

312 reguläre Projektionen wurden nach exakten kanonischen IDs, vollständigen Namen-/Beschreibungssentinels und verbotenen Keys durchsucht. Keine dieser Prüfungen fand einen Leak. Ein zusätzlicher Fall mit person:killer, event:murder, evidence:murder-weapon und location:hidden-basement gab diese Strings ebenfalls nicht aus.

EXPERIMENTALLY VERIFIED

Gesuchte Information	Beobachtung	Grenze der Aussage
Raw IDs, Namen, Descriptions	Kein Treffer in 312 Projektionen plus Speaking-ID-Probe	Nur der getestete aktuelle Projektionspfad; keine Aussage über künftige UI-/Logging-Adapter.
truth / truthHash / solutionHash	Kein objektives truth-Feld und keine Hashes	Knowledge-Stance enthält seinen faktiven Boolean-Wert bewusst.
Resolutions, requiredConclusions, Secrets	Keine entsprechenden Daten/Keys	Ein explizit als Attitude freigegebener Geheimnis-Claim erscheint trotzdem. Secret ist kein Zugriffsrecht.
Provenance, acquiredAt	Nicht ausgegeben	asOf und Claim-Zeit at werden absichtlich ausgegeben.
Hidden entities	Hinzufügen ungesehener Person/Location änderte Ausgabe nicht	Entitäten innerhalb freigegebener Claims sind per Definition sichtbar, auch ohne Awareness-Eintrag.
Event-/Evidence-Awareness	Nur lokale Referenz, keine automatische Link-/Teilnehmerexpansion	Ein bekannter Event verrät keine vollständige Ereignisbeschreibung.

EXPERIMENTALLY VERIFIED

Gesuchte Information	Beobachtung	Grenze der Aussage
Stale Truth-/Solution-Kontext	CONTEXT_BINDING_MISMATCH; keine Context-Ausgabe	Setzt gültige geparste Snapshots als API-Vorbedingung voraus.
Neun Claim-Kinds	Alle neun Formen in gemeinsamer Projektion bestätigt	role, value, Mechanism sind Inhalt freigegebener subjektiver Claims.

VERIFIED FROM CODE

Wichtig: Die Projektion ist kein wahrheitsblinder öffentlicher Dialog. knowledge:true oder knowledge:false verrät wegen Faktivität die Wahrheit eines freigegebenen Claims. Auch die Stance-Art knowledge selbst ist epistemische Information. Im erlaubten Kontext ist das beabsichtigt; für einen Spieler- oder LLM-Dialogadapter muss entschieden werden, welche Teile weitergegeben werden.

EXPERIMENTALLY VERIFIED

Ein Reihenfolgekanal bleibt: Handles sind die UTF-16-Ränge sichtbarer Raw-IDs pro Kategorie. Eine reine Umbenennung sichtbarer IDs veränderte den self.index. Das verrät relative Reihenfolge und Anzahl sichtbarer Referenzen, nicht automatisch unsichtbare Entitäten oder die Zeichenfolge „killer“. Keine stabilen Welt-IDs aus diesen Handles ableiten.

PRODUCT JUDGMENT

Produktbewertung: GO für die überprüfte interne Informationsgrenze, GO WITH CHANGES für einen zukünftigen öffentlichen Adapter. Keine aktuellen Leak-Funde rechtfertigen einen Umbau auf permanente PlayerRefs innerhalb dieses Audits. Ein UI muss lokale Indizes ohnehin auf freigegebene Anzeigeinformationen abbilden.

11. Complexity

EXPERIMENTALLY VERIFIED

Gemessen wurden vier Größen mit jeweils drei Graph-/Präsenzmustern: chain, dense (alle früheren Events als Ursache) und shared (alle Personen in jedem Event, zeitlich getrennt). Pro Operation vier Warmups und 15 Messungen; unten stehen Mediane in Millisekunden. Alle zwölf Welten hatten null Semantic-Findings. Die detaillierten Minimum-/Maximum- bzw. p95-Werte stehen im Rohdatensatz.

EXPERIMENTALLY VERIFIED

Umgebung: Node v24.19.0, Linux x64, AMD EPYC 9V74; Zod 4.6.5. Keine Smartphone-Messung, kein Durchsatz-SLA und kein Versuch, aus wenigen Punkten einen Exponenten statistisch zu schätzen. Bei 15 Samples entspricht der ausgewiesene p95 praktisch dem größten Messwert.

EXPERIMENTALLY VERIFIED

Größe / Muster	P / E	Kanten	Truth parse	Semantic	Truth hash	Solution parse
small / chain	5 / 10	9	0.228	0.022	0.145	0.313
small / dense	5 / 10	45	0.254	0.024	0.151	0.382
small / shared	5 / 10	9	0.215	0.025	0.133	0.288
medium / chain	10 / 30	29	0.518	0.088	0.348	0.534
medium / dense	10 / 30	435	0.703	0.142	0.410	0.726

EXPERIMENTALLY VERIFIED

Größe / Muster	P / E	Kanten	Truth parse	Semantic	Truth hash	Solution parse
medium / shared	10 / 30	29	0.528	0.172	0.323	0.498
large / chain	25 / 100	99	1.700	0.786	1.150	1.421
large / dense	25 / 100	4950	3.050	2.395	1.910	2.523
large / shared	25 / 100	99	2.202	2.865	1.320	1.533
stress / chain	50 / 250	249	3.801	3.024	2.929	3.141
stress / dense	50 / 250	31125	13.921	29.047	7.891	9.449
stress / shared	50 / 250	249	6.806	28.775	4.530	4.680

EXPERIMENTALLY VERIFIED

Bei 250 Events liegt Semantic Validation in der einfachen Kette bei rund 3 ms, im dichten Kausalgraphen und bei vielen gemeinsamen Teilnehmern bei rund 29 ms. Das ist in diesem Build-Time-Lab noch praktikabel. Die Größenordnung allein rechtfertigt weder „Performanceproblem im Spiel“ noch „beliebig skalierbar“.

VERIFIED FROM CODE

Codestelle	Wachstum aus Code	Autorenrelevanz
Präsenzkonflikte (C2:87)	Summe $k(\text{entity})^2$; bei P Personen in E Events $O(P \cdot E^2)$	Viele wiederholte Teilnehmer sind teurer als viele voneinander unabhängige Events.
Causal reachability (C2:185)	$O(V \cdot (V+E))$; bei dichtem DAG $O(V^3)$, Speicher $O(V^2)$	250 Events mit 31.125 Kanten zeigen erwartbar stärkeren Anstieg.
Doppelte Claim-Gruppen (C2:112)	Quadratische Paare innerhalb gleicher Claims	Viele gleichartige Alias-Propositionen ohne neuen Informationsgehalt vermeiden.
Negative personAt gegen Events	Für jede solche Proposition Scan der Events, darin Teilnehmerlookup	Sehr viele Alibi-Ticks können teuer werden; keinen Tick-für-Tick-Dump erzeugen.
Hash + Bindung (C3-C7)	Kanonische Sortierung und Weltserialisierung; wiederholt pro Schema/Projektion	NPC-Parse und Projektion bezahlen erneut den Truth-Hash, auch bei kleinem Snapshot.

PRODUCT JUDGMENT

Minimum: zuerst repräsentative Golden Cases und Größenbudgets. Ein späterer Befund kann indexierte Präsenzprüfung, lineare SCC-Erkennung und auf immutable Snapshots begrenztes Hash-Caching rechtfertigen. Keine Optimierungsimplementierung und keine Änderung am Hashprofil für den Slice erforderlich.

12. Canonicalization

EXPERIMENTALLY VERIFIED

384 Permutationsversuche (128 Fälle × 3 Varianten) permutierten rekursiv Arrays und Objektproperties und parsten die Resultate erneut. Truth- und Solution-Hashes blieben in allen Versuchen gleich. 15 weitere Canonicalization-Reihen untersuchten Text, Gruppierung, Nullwerte, Revision, Duplikate und große Sammlungen; dazu kamen zwei fokussierte ID-/Unicode-Probes.

EXPERIMENTALLY VERIFIED

Operation	Ergebnis	Einordnung
Array-/Property-Permutation	Alle Truth-/Solution-Hashes gleich	Gewollte Set-Semantik aller heutigen Arrays.
Unicode NFC vs. NFD	Hash verschieden	Strings werden ausdrücklich nicht Unicode-normalisiert. Gleiche Anzeige ist nicht gleiche Identität.
Echter Zeilenumbruch vs. zwei Zeichen Backslash+n	Hash verschieden	Escape-Unterscheidung korrekt erhalten.
Whitespace im Text	Hash verschieden	Kein Trim; Textänderungen invalidieren Binding.
Emoji, Steuerzeichen und Escapes bei gleichen Werten	Hash gleich	JSON-Escaping erzeugt deterministische Bytes.
Zwei verschiedene unpaired surrogates	Hash verschieden	JSON-Serialisierung schützt diese Zeichen vor UTF-8-Ersatzzeichen-Kollision.
Zusätzlicher gleichartiger Proposition-Alias	Hash verschieden	Neue ID ist zusätzlicher Inhalt; semantische Redundanz wird nicht weggedupliziert.
Doppelte identische Entity-ID	Vor Hash im Parser abgelehnt	Kein Anlass für eine Hash-Kollisionsinterpretation.
Andere Secret-Gruppierung	Hash verschieden	Array-Sortierung zerstört keine Objekt-/Containerzuordnung.
Andere Rollenverteilung auf Personen	Solution-Hash verschieden	Gleiche Rollenmenge global ist nicht dieselbe Verantwortungsstruktur.
complete↔partial; roles:null; Revision	Solution-Hash verschieden	Fachlicher Status und Version gehören zur Identität.
Instant @10 vs. Intervall [10,11)	Truth-Hash verschieden	Unterschiedliche Repräsentation; keine temporale Äquivalenznormalisierung.
250 Events / 31.125 Kanten permutiert	Truth-Hash gleich	Große set-artige Sammlungen bleiben stabil.
128 verschiedene Canonical-Truth-Texte	Keine gleiche Digest bei verschiedenem Text beobachtet	Kein kryptographischer Beweis; begrenzte Stichprobe.

VERIFIED FROM CODE

Die implementierten Profile `forge-case-c14n-v1` und `forge-solution-c14n-v1` sind explizite Domainprofile, kein RFC-8785/JCS. Unterschiedlicher Text, Revision oder ID darf absichtlich einen neuen Hash erzeugen, auch wenn ein Mensch dieselbe Geschichte erkennt. Gruppierung und Nullwerte bleiben erhalten. Kein unerwarteter Gleichhash-Bug wurde in diesem Korpus gefunden.

PRODUCT JUDGMENT

Für LLM-Authoring müssen kanonische IDs und Textfassungen nach Validierung eingefroren werden. „Nur die Formulierung verbessert“ ist eine neue Snapshot-Fassung und verlangt Neubindung. Künftige wirklich geordnete Arrays dürfen nicht unbemerkt in das heutige Alles-ist-Set-Profil aufgenommen werden.

13. Five Templates

PRODUCT JUDGMENT

Die folgenden Vorlagen sind bewusst fachliche Schreibrezepte, kein neuer Generatorcode. Sie verwenden die erfolgreich getesteten Strukturen. Jede Vorlage verlangt eine getrennte menschliche Prüfung der Schlusskette, weil die aktuellen Parser keine spielbare Beweisführung garantieren.

T1 - Simple culprit

PRODUCT JUDGMENT

Baustein	Authoring-Vorlage
Entities	3 Personen (Täter A, Betroffener B, Vergleichsperson C), 2 Orte, 1 Item.
Events	E0 Vorbereitung mit A; E1 relevantes Ereignis mit B und ggf. A; E2 Alibi von C. Zeitliche Überlappung nur mit konsistenten Ortsdaten. E1 kann E0 direkt als Ursache führen.
Propositionen	A nimmt an E0 teil; Item ist in E0; C ist zum relevanten Tick am anderen Ort. Anwesenheit von A bei E1 nur behaupten, wenn sie tatsächlich modelliert ist.
Evidence	Mindestens zwei separat prüfbare Hinweise an Präsenz-/Item-Propositionen; davon ein Hinweis, der eine konkrete Alternativhypothese schwächt. Keine behauptete technische Schuldableitung.
Solution	E1: complete, A <code>direct_actor</code> ; mindestens A-verantwortlich true. Falls die spätere Antwortform Ausschlüsse verlangt: C-verantwortlich false explizit aufnehmen.
NPC knowledge	Ein NPC kennt eine wahre Präsenz; ein anderer glaubt einen falschen Alibi-Claim. Knowledge nur für Propositionen.
Typische Fehler	Mörder aus Anwesenheit ableiten; <code>eventHasItem</code> als „Waffe benutzt“ lesen; negatives <code>Required-Literal</code> unter <code>partial</code> erzwingen.

T2 - Planner + actor

PRODUCT JUDGMENT

Baustein	Authoring-Vorlage
Entities	4 Personen (Planer A, Actor B, Betroffener C, Zeuge D), 3 Orte, 1-2 Items.
Events	E0 Planung/Vorbereitung, E1 Handlung, E2 Folge. A muss bei E1/E2 nicht teilnehmen. Kanten bezeichnen unmittelbare Ursachen.
Propositionen	Belegte Teilnahme von A an E0, B an E1, Itempräsenz. Keine Verantwortung in ClaimSchema einschmuggeln.
Evidence	Hinweise auf getrennte Vorbereitungs- und Ausführungsschritte; Verknüpfung zur Schuld bleibt begründungsbedürftig.
Solution	Eine E2-Resolution mit A:planner und B:direct_actor. Separate Verantwortungs- und Rollen-Conclusions; beide Personen nur dann required, wenn die Antwortaufgabe beide verlangt.
NPC knowledge	D kennt Vorbereitung oder Ausführung; Schluss auf Planer als Belief. Planer kann denselben Claim bestreiten, ohne objektive Truth zu verändern.
Typische Fehler	Planer zum Teilnehmer des Tatorts machen; indirekte E0→E2-Kausalität als direkte Kante required setzen; A zweimal als assignment schreiben.

T3 - Accident / no responsible person

PRODUCT JUDGMENT

Baustein	Authoring-Vorlage
Entities	3 Personen als Betroffene/Zeugen, 2 Orte, 1 technisches Item.
Events	E0 natürlicher oder technischer Auslöser, E1 Folge, E2 Beobachtung; Akteur-freie Events dürfen leere participantIds haben.
Propositionen	Wer beobachtete welches Event, welches Item war vorhanden, welche Person war wo. Der Defekt selbst bleibt ohne neues Zustandsmodell eine Ereignisbeschreibung.
Evidence	Objekt- und Beobachtungshinweise; kein Evidence-Richtungstest beweist den Unfall automatisch.
Solution	E1: complete + assignments:[]; noPersonResponsible true. intent unintended oder not_applicable nach bewusster Fallentscheidung; mechanism ordinary. Menschlich verursachter Unfall braucht gegebenenfalls assignments.
NPC knowledge	Zeuge kennt objektive Beobachtung; ein anderer beschuldigt irrtümlich eine Person als Conclusion-Belief.
Typische Fehler	partial+[] als „niemand“ interpretieren; Unfall zwingend mit Schuldlosigkeit gleichsetzen; ordinary als typisierten Defekt lesen.

T4 - Multiple actors / one multi-role person

PRODUCT JUDGMENT

Baustein	Authoring-Vorlage
Entities	5 Personen, 3 Orte, 2 Items; 2-3 Verantwortliche und mindestens eine Vergleichsperson.
Events	Getrennte Vorbereitungsschritte, gemeinsames oder zeitversetztes Hauptereignis. Gemeinsame Teilnehmer benötigen im selben Zeitraum denselben Ort.
Propositionen	Präsenz je Arbeitsschritt und Item; redundante Claims sparsam halten.
Evidence	Hinweise, die Rollen unterscheiden, statt nur alle Verdächtigen pauschal mit dem Hauptereignis zu verbinden.
Solution	Eine assignment pro Person. Mehrrollen-Person mit Rollenliste, nicht mehrfacher assignment. Für ein abgeschlossenes Tätermengenrätsel complete; bei absichtlich offener Verantwortung partial plus nur determinierte requiredConclusions.
NPC knowledge	Mehrere subjektive Perspektiven; einer kennt Vorbereitung, einer Ausführung, einer glaubt eine überbreite Tätermengenhypothese.
Typische Fehler	Nur einen Täter required setzen, obwohl alle gefragt werden; extra Beschuldigten über spätere Subset-Wertung durchlassen; roles:null als keine Rolle deuten.

T5 - Supernatural / technical causal chain

PRODUCT JUDGMENT

Baustein	Authoring-Vorlage
Entities	3-5 Personen, 3 Orte, 2-3 Items. Übernatürliche Ursache als Event/Mechanism, nicht als ungeprüft neuer Entity-Typ.
Events	E0 und E1 als zwei direkte Ursachen von E2; E2 verursacht E3. Die Daten sagen nicht, ob E0 UND E1 notwendig sind oder jeder allein genügt.
Propositionen	Beobachtbare Präsenz-/Item-Claims an den Stufen; kein Freitext als typisierte Magie- oder Technikregel ausgeben.
Evidence	Hinweise auf einzelne Stufen; echte Interpretation von Reihenfolge und Mechanism wird redaktionell geprüft.
Solution	mechanism supernatural oder mixed; eventCausedEvent für direkte Kanten; causesComplete nur bei vollständiger Kantenliste. E3 kann keine menschliche Verantwortung oder bewusst zugewiesene Verantwortung haben.
NPC knowledge	Ein NPC kennt eine Beobachtung, interpretiert die Ursache aber falsch. Mechanism bleibt Conclusion-Belief/Uncertainty.
Typische Fehler	Transitive Kausalität required setzen; rückwärts gerichtete Kante; AND/OR-Semantik erfinden; Supernatural als Ausnahme von Ortskonsistenz behandeln.

14. Generation Pipeline

PRODUCT JUDGMENT

Kleinste experimentell tragfähige spätere Pipeline: Kandidateninhalt erzeugen; CaseTruth parsen; Semantic Validation zwingend ausführen; Truth-Hash deterministisch berechnen; Solution mit diesem Hash parsen; Solution-Hash berechnen; NPC-Snapshots erzeugen und binden; NPCs parsen und projizieren; Fall redaktionell prüfen; freigegebene Fassungen einfrieren. Ein zukünftiger Solvability-/Package-Schritt bleibt ausdrücklich außerhalb der heutigen Implementierung.

PRODUCT JUDGMENT

Schritt	LLM darf	Deterministisch / Gate	Bei Fehler
Kandidat	Entities, Events, Texte und vorgeschlagene Claims erzeugen	Typen, IDs, Referenzen, Zeitspannen parsen	Lokaler Reparaturauftrag mit Pfad; keine automatische Umdeutung der Story
Semantik	Ursachen und räumliche Dramaturgie vorschlagen	Alle Semantic-Findings müssen behandelt werden	Root-Cause plus betroffene IDs zeigen; nach Reparatur ganze Semantik erneut
Solution	Resolutions und erwünschte Antwortkriterien vorschlagen	Hash berechnen, bindend parsen, Required-Literale prüfen	undetermined nicht in false umdeuten; Autor muss Vollständigkeit bewusst wählen
NPCs	Stances und plausible Herkunft vorschlagen	Bindung, Erwerbsregeln, Knowledge-Faktivität, Duplikate	Snapshot lokal reparieren; Truth-Änderungen lösen Neubindung aller Abhängigkeiten aus
Projection	Nur geplante Freigaben und spätere Formulierungsvorschläge	Echten Projektor nutzen; Sentinel-/Allowlist-Checks	Ausgabe verwerfen; keinen Raw-Snapshot als Fallback ausgeben
Redaktion	Alternative Interpretationen beschreiben	Mensch bewertet Beweisbarkeit, Fairness, Vielfalt und falsche Fährten	Unklare Falllogik zurück ans Design; kein rein syntaktischer Retry
Späterer Pak etabschluss	Keine selbst erfundenen Signaturen/Hashwerte	Solvability, Zugriff und Package-Bindung erst nach ihrer Implementierung	Heute keine Behauptung eines validierten CasePackage

PRODUCT JUDGMENT

Retry-Vorschlag: höchstens zwei fokussierte Reparaturversuche je Fehlertyp, danach menschliche Entscheidung; die Zahl ist ein Produktbudget, kein gemessener optimaler Parameter. Ein Reparaturauftrag muss den bestehenden fachlichen Entwurf erhalten. Kein „findings leer“ durch Entfernen kritischer Evidence, falsches Umschalten auf complete oder Löschen schwieriger requiredConclusions.

VERIFIED FROM CODE

LLMs erzeugen Inhalte und Vorschläge. Parser, Hashes, Vergleichslogik, Negativkontrollen und Release-Gates bleiben deterministisch. Kein Runtime-LLM ist nötig oder in diesem Lab implementiert. Ein eingefrorener Satz gültiger Snapshots ist heute möglich; das ersetzt kein zukünftiges Spielpaket mit Save/Load- oder Solvability-Garantie.

15. Error Feedback

EXPERIMENTALLY VERIFIED

Echtes Beispiel	Qualität aus Autorensicht	Verbesserung ohne neues Domain-Feld
events[0].participantIds[1]: Unresolved reference "person:absent"	Gut: genaue Fundstelle und fehlende ID	Vorhandene Personen-IDs bzw. eindeutige Tippfehlerkandidaten anzeigen.
events[0].time.end: Interval requires start < end	Gut: reparierbarer lokaler Fehler	start/end zusätzlich als konkrete Werte anzeigen, keine automatische Zeitkorrektur.
requiredConclusions[1].value: required true but resolves to undetermined	Nützlich, aber Ursache liegt in anderer Struktur	Verlinken auf assignment.roles:null / partial / causesComplete:false.
attitudes[0]: Invalid input bei Conclusion knowledge	Schlecht im Kurzformat; verschachtelte Union enthält eigentliche Ursache	Union aufbereiten: „Conclusion erlaubt nur belief oder uncertain“.
redHerrings[0].misleadingPropositionId bei conclusion:-ID	Prefix-Fehler plus Unresolved-reference-Folgefehler	Beide Meldungen behalten, gleichen Primärfehler in UI bündeln.
SemanticFinding mit code + subjectIds + message	Inhaltlich gut, kein JSON-Pfad	ID→Datei/Pfad-Index daneben bereitstellen.
Geänderte Resolution-Event-ID	Ein Unknown event plus acht No resolution-Folgefehler im Beispiel	Ursache zuerst gruppieren, Folgefehler als abhängige Befunde markieren.
Stale truthHash + unbekanntes Event	Zuerst nur Binding-Fehler; nach Rebind werden Referenzfehler sichtbar	Mehrstufigen Diagnoseablauf erklären; nicht als plötzlich neuen Fehler verkaufen.
Same claim as "conclusion:c0"	Gute Vergleichsreferenz, aber second-path-only	Beide Conclusion-Pfade nebeneinander anzeigen.
PERSON_LOCATION_CONFLICT mehrfach	Ein hinzugefügtes kollidierendes Event kann mehrere Befunde erzeugen	Konfliktquellen und betroffene Zeitstelle clustern; keine eindeutigen Ursachen ungeprüft löschen.

VERIFIED FROM CODE

Strukturelle Zod-Fehler besitzen Codes, Pfade und Meldungen; viele fachliche Parserfehler teilen allerdings den unspezifischen code:custom. Semantic-Findings besitzen stabile FindingCodes, aber keine Pfade. Eine einheitliche maschinenlesbare Diagnose kann diese Unterschiede zunächst im CLI aufbereiten; ein Umbau aller Domain-Schemas ist dafür nicht erforderlich.

PRODUCT JUDGMENT

Adressat	Einschätzung	Beleg / Grenze
Mensch	GO WITH CHANGES	Lokale Fehler gut reparierbar; Cross-References und undetermined brauchen Tabelle/Leitfaden.
Claude	GO WITH CHANGES als erwartbare Nutzung	Strukturierte Pfade helfen; keine tatsächliche Claude-Reparaturrate gemessen.

PRODUCT JUDGMENT

Adressat	Einschätzung	Beleg / Grenze
Codex	GO WITH CHANGES als erwartbare Nutzung	Gleiche Einschränkung; Quellcode-Lesen kann die Semantik erklären, ersetzt kein Erfolgsmaß.
Build-Time-Generator	GO WITH CHANGES	Geordnete Gates und begrenzte Retries sind möglich; custom-Messages nicht als dauerhaftes Protokoll parsen.

UNKNOWN

UNKNOWN: Wie viele Reparaturschleifen ein reales Modell benötigt, wie oft es eine Fallidee beschädigt und welche Fehlermeldung die Erfolgsquote erhöht. Das würde einen eigenen Agentenvergleich mit eingefrorenem Prompt- und Aufgabenbestand verlangen.

16. Minimum Authoring Tool

PRODUCT JUDGMENT

Option	Nutzen	Kosten / Grenze	Empfehlung
A. TypeScript/JSON von Hand	Sofort nutzbar; TS kann Shapes statisch unterstützen	Cross-File-Binding und Errors bleiben Handarbeit; TS-Autorenfiles sind ausführbarer Code	Geeignet für zwei erste Entwickler-Fixtures, nicht als ungeprüfter LLM-Import.
B. YAML/JSON + Validator CLI	Datenorientiert; editierbar; bestehende Domain-Parser nutzbar	YAML fügt eigene Normalisierungs-/Typfallen hinzu	JSON zuerst; YAML erst bei belegtem Nutzerbedarf.
C. Kleines CLI mit guten Diagnosen	Ein durchgehender Befehl für alle Gates, Hashes und Pfad/ID-Auflösung	Kleine, klar begrenzte Tooling-Arbeit	MINIMUM für wiederholbares Authoring vor größerer Fallproduktion.
D. Grafischer Editor	Timeline/Graph/Referenzen sichtbar, bequem bei vielen Fällen	Hoher Entwicklungs- und Pflegeaufwand vor stabilem Workflow	LATER; nicht vor dem ersten Slice begründen.

PRODUCT JUDGMENT

Empfohlen ist B in der kleinsten brauchbaren C-Ausführung: JSON-Dateien plus ein dünner Validator-/Binding-CLI-Wrapper. Er ruft bestehende Parser auf, zeigt Phase, Code, Pfad und IDs, berechnet Hashes und liefert einen eindeutigen Exit-Status. Keine graphische Oberfläche, kein DSL, kein Runtime-LLM und kein Produktionsgenerator. Ein solcher Wrapper wurde in diesem Auftrag ausdrücklich nicht als Produkt implementiert.

PRODUCT JUDGMENT

Vor einem einzelnen handgebauten Vertical Slice braucht ihr keinen eigenständigen Editor. Vor wiederholtem Authoring oder LLM-Import braucht ihr jedoch einen verlässlichen einen Weg durch alle Validierungsstufen. Der experimentelle Scratch-Runner zeigt dessen notwendige Reihenfolge, ist aber nicht das zukünftige Autorenwerkzeug.

17. Ten Golden Cases

PRODUCT JUDGMENT

Die folgenden kleinen Fälle sind ein vorgeschlagener langfristiger Domain-Korpus. Sie sind fachliche Spezifikationen, noch keine zehn zusätzlich implementierten Produktions-Fixtures. Die jeweils zugrunde liegenden schwierigen Strukturen wurden im Lab geprüft. Erwartete Ergebnisse beziehen sich auf heutige Parser; zukünftiges Verdict-/Solvability-Verhalten wird gesondert benannt.

G01 - Der einzelne Verantwortliche

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Geschlossener Normalfall und Negativkontrolle für falsche Täterschaft.
Struktur	3 Personen, 2 Orte, 3 Events; A direct_actor, complete; A-responsible true und C-responsible false required.
Erwartung	Truth PASS; Semantic 0; Solution PASS; falsches Required-Literal wird abgelehnt.
Langfristiger Wert	Kleinster Ende-zu-Ende-Fall für Binding und expliziten Ausschluss; spätere Verdict-Engine muss A+C als falsch behandeln, sofern exakte Antwort verlangt wird.

G02 - Zwei Beteiligte, ein Unschuldiger

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Multi-Actor-Verantwortung ohne Shotgun-Abkürzung.
Struktur	4 Personen, 2 Orte, 4 Events; A und B direct_actor, C ausgeschlossen; separate Required-Literale.
Erwartung	Truth PASS; Semantic 0; Solution PASS; Weglassen von B wird heute nicht als Spielerantwort geprüft, da Verdict fehlt.
Langfristiger Wert	Regression für vollständige Tätermenge und zusätzliche falsche Beschuldigung in der späteren Verdict-Schicht.

G03 - Der abwesende Planer

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Verantwortung ist nicht Anwesenheit.
Struktur	4 Personen, 3 Orte, 4 Events; A bei Vorbereitung, B bei Ausführung; A planner, B direct_actor für Folgeevent.
Erwartung	Semantic 0 trotz Abwesenheit des Planers vom Folgeevent; Rollen-Required-Literale PASS.
Langfristiger Wert	Verhindert künftige falsche Kopplung von responsibility und participantIds.

G04 - Der unabsichtliche Unfall

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	intent unintended unabhängig von Täter-Narrativ.
Struktur	3 Personen, 2 Orte, 3 Events; ein unbeabsichtigter menschlicher Eingriff und Folge; A responsible, intent unintended.
Erwartung	Truth/Solution PASS; beide Conclusions responsibility und unintended determinieren true.
Langfristiger Wert	Verhindert Gleichsetzung „Unfall = niemand verantwortlich“ oder „verantwortlich = absichtlich“.

G05 - Niemand menschlich verantwortlich

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Explizite leere vollständige Verantwortung.
Struktur	3 Personen als Zeugen, 2 Orte, 3 Events; technische oder übernatürliche Ursache; complete+[], noPersonResponsible true.
Erwartung	PASS; Umstellung auf partial macht dieses Required-Literal undetermined und wird abgelehnt.
Langfristiger Wert	Klärt leere Menge versus offene Unkenntnis und unterstützt Fälle ohne Täterperson.

G06 - Bekannter Helfer, offene Hauptverantwortung

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Partial-Verantwortung und geschlossene Rollenliste.
Struktur	4 Personen, 2 Orte, 3 Events; A facilitator unter partial; A responsible true; outsider-Verantwortung nicht required.
Erwartung	PASS; outsider=false required wird abgelehnt; A direct_actor=false ist bei bekannter Rollenliste trotzdem bestimmbar.
Langfristiger Wert	Schützt Three-valued-Semantik vor unbeabsichtigter Closed-world-Ausweitung.

G07 - Zwei Ursachen, drei Stufen

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Direkte Kausalität, Reihenfolge und Zyklen.
Struktur	3 Personen, 3 Orte, 5 Events; A und B→C, C→D; causesComplete bewusst festgelegt.
Erwartung	Semantic 0; A→D als direkt required:true abgelehnt; Rückwärtskante erzeugt Zeitfehler, Cycle erzeugt CAUSAL_CYCLE.
Langfristiger Wert	Fixiert direkte statt transitive Conclusion-Semantik und die Grenze zu AND/OR-Kausalmodellen.

G08 - Der überzeugte Irrtum

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	False belief von false knowledge trennen.
Struktur	3 Personen, 2 Orte, 3 Events; falsche Präsenzproposition; NPC belief:true; zweiter Snapshot knowledge:false.
Erwartung	Beide Snapshots PASS; knowledge:true zu derselben falschen Proposition REJECT.
Langfristiger Wert	Verhindert automatische Wahrheitssynchronisierung von Beliefs und falsche Interpretation von knowledge:false.

G09 - Widersprüchliche Zeugen

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Subjektive Widersprüche ohne inkonsistente objektive Welt.
Struktur	4 Personen, 2 Orte, 3 Events; zwei NPCs vertreten gegensätzliche Beliefs zur Präsenz; optional ein NPC glaubt zwei unvereinbare räumliche Claims.
Erwartung	Truth Semantic 0; subjektiv unvereinbare Stances in getrennten Snapshots bzw. zu verschiedenen Claims erlaubt; Alias-Doppelattitude REJECT.
Langfristiger Wert	Hält Glaubwürdigkeit/Verhörlogik von objektiver Semantic Validation getrennt.

G10 - Wahrer Köder, falscher Schluss

PRODUCT JUDGMENT

Aspekt	Golden-Case-Spezifikation
Zweck	Grenze des alten RedHerring-Modells dauerhaft sichtbar machen.
Struktur	4 Personen, 2 Orte, 4 Events; wahrer Präsenzhinweis, separate falsche Täter-Conclusion, NPC belief:true zur falschen Conclusion.
Erwartung	Fakt und Belief einzeln PASS; conclusion:-ID als misleadingPropositionId REJECT; kein heutiger Beweisgraph für den Interpretationsübergang.
Langfristiger Wert	Schützt vor Schuld-als-Anwesenheit-Workarounds und wird Prüfstein für spätere Conclusion-Misdirection.

18. Model Changes

PRODUCT JUDGMENT

Es wurde kein zwingender Schema- oder Hashing-Bug gefunden, der vor einem eng begrenzten, handgebauten Slice neue Domain-Felder erfordert. Es gibt jedoch zwingende Integrations- und Scope-Entscheidungen. MUST bedeutet hier eine Voraussetzung für verlässliches Authoring des Slice, keine ungefragte Implementierungsfreigabe.

PRODUCT JUDGMENT

Priorität	Änderungswunsch / Befund	Empfehlung und Abgrenzung
MUST BEFORE VERTICAL SLICE	AF-01: Ein verbindlicher Import-/Validierungsweg	Truth parse → Semantic → Binding → Solution → NPC → Projection; kein Freigabeweg über parseCaseSolution allein. Tooling/Gate.
MUST BEFORE VERTICAL SLICE	AF-02: Antwortumfang im Slice konkret festlegen	Exakte Täter-/Rollenmenge oder konkrete Required-Literale; zusätzliche falsche Antworten und partial ausdrücklich behandeln. Vorhandenes Verdict nicht vortäuschen.
MUST BEFORE VERTICAL SLICE	AF-03: Fall-spezifische Beweis-/Freigabepfung	Für den ersten Fall jede erforderliche Erkenntnis und Gegenhypothese redaktionell prüfen; Projection nicht direkt als öffentlicher Dialog behandeln. Kein generischer Solvability-Validator im Auftrag.
SHOULD	AF-04: Diagnose-Wrapper	Stufe, Pfad, Subject-IDs, Primär-/Folgefehler und Werte-Vorschau einheitlich darstellen; keine Message-Text-Parsing-Abhängigkeit.
SHOULD	AF-05: Required-Conclusion-Statusvorschau	true/false/undetermined mit begründender Resolution anzeigen. Prüfen, ob ein kleines öffentliches Diagnose-API Doppelimplementierung verhindert.
SHOULD	AF-06: Autorenleitfaden + Golden-Korpus	partial/complete, roles:null, direkte Ursachen, Half-open-Zeit und NPC-Faktivität dokumentieren; fünf Templates und zehn Cases übernehmen.
SHOULD	AF-07: Unverbindliche Authoring-Warnungen	Unverbundene RedHerrings, redundante Claims, schwache Evidence-Abdeckung und un plausible Provenance als Warnungen, nicht pauschale Domain-Verbote.

PRODUCT JUDGMENT

Priorität	Änderungswunsch / Befund	Empfehlung und Abgrenzung
SHOULD	AF-08: Deterministische Hash-/ID-Pflege	Autor-Dateien von gebundenen Snapshots trennen; Hashes berechnen; kanonische ID-Änderung als Inhaltsänderung behandeln.
LATER	AF-09: Conclusion-basierte Misdirection	Erst anhand G10 einen Interpretations-/Begründungsvertrag entwerfen. Nicht das alte Proposition-RedHerring-Feld überladen.
LATER	AF-10: Typisierte Besitz-/Schadens-/Zustandssemantik	Nur wenn Diebstahl-/Defektfälle diese Prüfung tatsächlich benötigen. Kein universales Rätselontologie-Projekt.
LATER	AF-11: Evidenzzugang, Lösbarkeit, Eindeutigkeit, Package	Eigenständige fehlende Produktfähigkeiten; „Validator grün“ darf sie nicht ersetzen.
LATER	AF-12: Performance und GUI	SCC/Präsenzindex/Hash-Caching bei belegter Größenlast; grafischen Editor erst nach stabilem Autorenworkflow.
DO NOT CHANGE	Verantwortung unabhängig von physischer Teilnahme	Erforderlich für Planer, Helfer, Fernwirkung; kein Anwesenheitszwang einbauen.
DO NOT CHANGE	partial / complete, roles:null, causesComplete	Unbestimmtheit fachlich bewahren; keine automatischen false-Werte.
DO NOT CHANGE	Faktive Proposition-Knowledge und freie Beliefs	Unterschied bleibt; widersprüchliche subjektive Welt nicht mit Truth-Validator „reparieren“.
DO NOT CHANGE	Heutiges Hashprofil und lokale Projektion	Keine Unicode-Normalisierung oder permanente PlayerRefs nur wegen Authoring-Komfort einführen.
DO NOT CHANGE	Evidence-Richtung darf Wahrheit verfehlen	Irreführung muss möglich bleiben; supports nicht automatisch als wahr prüfen.

19. GO/NO-GO Matrix

PRODUCT JUDGMENT

Bereich	Urteil	Begründung / Bedingung
CaseTruth authorability	GO WITH CHANGES	Referenzsicher und vielfältig; CLI-Führung für Timeline/Claims, eingeschränkte Objektzustände.
CaseSolution authorability	GO WITH CHANGES	Multi-Actor und offene Verantwortung sauber; Required-Auswahl und späteres Antwortformat müssen sichtbar sein.
NPC Knowledge authorability	GO WITH CHANGES	Breite getestete Stances; Provenance und Conclusion-Knowledge-Grenze benötigen Autorenführung.
Hash/binding model	GO	Permutationsstabil, gruppierungssensitiv, Stale-Bindings erkannt; Hashberechnung durch Tooling.
Validator feedback	GO WITH CHANGES	Lokale Pfade gut; Union-/Cross-Reference-/undetermined-Diagnosen brauchen Aufbereitung.
Case diversity	GO WITH CHANGES	16 Familien als Fakten-/Solution-Gerüste; narrativer und lösungslogischer Variantenreichtum nicht nachgewiesen.
False-lead support	GO WITH CHANGES	Falsche Präsenz/Beliefs gehen; komplexe Conclusion-Misdirection fehlt. Für automatisch abgesicherte komplexe False Leads: NO-GO.
Multi-actor support	GO WITH CHANGES	Assignment-/Rollenmodell funktioniert; exakte Spielerantwort und Shotgun-Verhalten erst später prüfbar.
Build-time LLM generation	GO WITH CHANGES	Als kontrolliertes Erzeugen und Reparieren mit deterministischen Gates und menschlicher Freigabe. Unbeaufsichtigtes Erzeugen veröffentlichbarer Fälle: NO-GO.

PRODUCT JUDGMENT

Kein pauschales „Modell untauglich“: Die scharf definierten Grenzen verhindern echte Widersprüche und lassen erstaunlich viele Antwortschlüssel zu. Kein pauschales „Generator bereit“: Der entscheidende Übergang von Indizien zu fair herleitbaren requiredConclusions ist im heutigen main nicht bewiesen. Das ist der wichtigste verbleibende Produktnachweis.

20. Exact Next Actions

PRODUCT JUDGMENT

Nr.	Konkrete Folgeaktion	Abnahmekriterium
1	Einen dünnen JSON-Validator-/Binding-CLI-Auftrag erstellen	Ein Befehl führt alle vorhandenen Gates aus, berechnet Hashes und liefert Phase+Pfad+IDs. Semantisch ungültige Truth darf nie als freigegebener Fall enden.
2	Die zehn Golden-Case-Spezifikationen in einen kleinen Domain-Testkorpus überführen	Je Fall positive Erwartung plus gezielter Negativzwilling; separate Erwartungen für World, Solution, NPC und späteren Verdict dokumentiert.
3	Für den ersten Slice einen konkreten Fall mit exakter Antwortform und Indizienweg festlegen	Jede erforderliche Erkenntnis erhält eine menschlich geprüfte Herleitung; zusätzlicher Unschuldiger, fehlender Mittäter und undetermined haben definierte Ergebnisse.
4	Fünf Templates und einen kurzen Semantik-/Fehlerleitfaden in den Authoring-Prozess übernehmen	Autoren können partial, null, direkte Ursache, Evidence-Richtung und Knowledge/Belief ohne Quellcode-Nachschlagen unterscheiden.
5	G10 als begrenzten Preflight für Conclusion-Misdirection ausarbeiten	Entscheidung: Slice nutzt bewusst nur Proposition-Fährten oder bekommt später einen eng definierten Interpretation-Vertrag. Keine Umcodierung von Schuld in Präsenz.

VERIFIED FROM CODE

Keine dieser Folgeaktionen wurde am kanonischen Repository umgesetzt. Es wurden keine Aufgaben, PRs, Kommentare oder Reviews dort angelegt. Der Auftrag endet mit diesem Bericht und den reproduzierbaren Scratch-Ergebnissen.

21. Experimental Appendix

A. Reproduktions- und Integritätsnachweis

VERIFIED FROM CODE

Quellstand: Commit 3d7545d843883418348004e68717399a64da7a7d. Start-Read von refs/heads/main per GitHub-Connector; Tree-ID mit lokaler Git-Objektdatenbank abgeglichen. Archivierung ausschließlich mit git archive des fixierten Commits. Der Appendix enthält nur die relevanten unveränderten Domain-Quellen, Domain-Tests/Fixtures und Paketmetadaten; keine Git-Credentials oder node_modules.

```
cd mystery-authoring-lab
npm ci --ignore-scripts --prefix source
node experiments/stress-lab.mjs
node experiments/focused-probes.mjs
node experiments/recheck-overlap.mjs
# results/*.json und results/*.log enthalten die Ergebnisse
```

EXPERIMENTALLY VERIFIED

Node >=24 für das direkte Ausführen der TypeScript-Quelldateien ohne zusätzlichen Loader verwenden. Die Paketdatei erlaubt Node >=22, diese Laboranleitung ist enger an die tatsächlich verwendete Laufzeit gebunden. Das bestehende Lockfile bindet Zod 4.6.5, Vitest 5.0.3 und TypeScript 7.0.2. Der ausgeführte Lauf nutzte vorhandene Dependencies dieser exakten Versionen über einen lokalen Scratch-Link; es wurde kein Paketmanifest verändert.

B. Codebelege am fixierten Commit

VERIFIED FROM CODE

ID	Pfad	Relevante Stellen
C1	src/domain/case-truth.ts	IDs 9-29; TimeSpan 70; ClaimSchema 127; Evidence/RedHerring 159-194; Referenzen 235; parse 338
C2	src/domain/case-semantics.ts	Grenzen 3-9; FindingCodes 12; Präsenz 87; Claims 112; Kausalität 185; Entry 231
C3	src/domain/case-solution.ts	Enums 27-34; Resolution 63; Conclusions 78; Three-valued-Semantik 126; Binding 160; Parser 294
C4	src/domain/case-truth.identity.ts	Profil 4-11; Canonicalization 17; Hash 35
C5	src/domain/case-solution.identity.ts	Profil 4-11; Canonicalization 17; Hash 35
C6	src/domain/npc-knowledge.ts	Stances 50-79; Binding 135; Acquisition 167; Faktivität/Claim-Duplikate 207; Parser 301
C7	src/domain/npc-knowledge.projection.ts	Allowlist 27; Binding 76; Claim-Konstruktion 113; Stances 141; sichtbare Menge/Handles 152ff.
C8	forge/BASELINE-STATUS.md	Mystery Included / Not included; historischer TASK-0005-Review ist nicht aktueller Implementierungsstand

C. Ergebnisdateien

EXPERIMENTALLY VERIFIED

Datei	Inhalt
source-manifest.json / domain-integrity.json	Commit, Tree, Erhebungsmethode und unveränderte Git-Blob-/SHA-256-Werte
results/summary.json	Finale Summen, Größenabdeckung, FindingCode-Häufigkeiten und Umgebung
results/candidates.json	Alle 200 Truth-/Solution-Kandidaten einschließlich der 72 absichtlich defekten Varianten
results/overlap-candidates.json / overlap-funnel.json	128 zusätzliche Ko-Lokalisierungsvarianten; 6 reale Generatorfehler, mit recheck-overlap.mjs erneut bestätigt
results/funnel.json	Stoppstufe, exakte Zod-Issues, Semantic-Findings und erfolgreiche Hashes je Kandidat
results/mutations.json	528 unabhängige Truth-/Solution-Mutationen; eine Zeile pro Seed und Operator
results/npc-results.json	576 NPC-Ergebnisse einschließlich der Fehlerpfade
results/npc-projections.json	312 akzeptierte Eingabesnapshots samt echter Projection-Ausgabe
results/leakage.json	312 Sentinel-/Key-Scans, Speaking-ID-Fall, Hidden-Entity-Vergleich, Stale-Kontexte
results/multi-actor.json	120 Literalprüfungen mit beiden Boolean-Werten
results/false-leads.json	9 RedHerring-/Evidence-Strukturprobes; interpretative Verknüpfung ausdrücklich nicht bewiesen
results/canonicalization.json	399 Canonicalization-Reihen einschließlich 384 Permutationspaaren
results/benchmark.json	12 Größen-/Musterkombinationen × 6 Operationen mit 15 Messungen nach Warmup
results/special-probes.json	5 Gate-/Feedback-/Lösbarkeitsgrenzen, darunter Null-Evidence und isolierter Zyklus
results/focused-probes.json	28 zusätzliche Probes mit Sollresultat; 0 unerwartete Ergebnisse
results/baseline-verification.json	452 bestandene vorhandene Mystery-Tests; kein Forge-Testurteil
experiments/stress-lab.mjs / focused-probes.mjs	Scratch-Generator und gezielte Probes; kein Produktwerkzeug
REPORT.html / report-data.json	Lesbare elektronische Fassung und strukturierte Berichtsinhalte

D. Coverage, Bias und nicht behauptete Ergebnisse

EXPERIMENTALLY VERIFIED

- 176 schema-gültige Truth-Kandidaten; davon 152 ohne Semantic-Findings und 128 vollständig akzeptierte Truth-/Solution-Paare. Die Zahlen sind nicht austauschbar.
- Alle acht implementierten FindingCodes wurden in den Mutationen ausgelöst. Dies ist Code-/Fehlerklassen-Coverage, keine Vollständigkeitsgarantie des Domain-Modells.

- 22 Mutationstypen auf 24 Fällen; NPC-Negativvarianten separat. Die Tests mutieren Eingaben, nicht den Implementierungscode; dies ist kein Mutation-Score der Produktions-Test-Suite.
- Die Hauptserie enthält überwiegend getrennte Ereigniszeiten. Half-open-Ränder, reine Zeitumkehr, isolierter Zyklus und überlappende Ursachen sind fokussierte Ergänzungen.
- Keine realen LLM-Aufrufe, keine Autoren-Benutzerstudie, kein spielbarer Vertical Slice, kein Solvability-Beweis und keine live getestete Verdict-Engine.
- Hash-Tests prüfen Metamorphie und Repräsentationsunterschiede. Eine endliche Stichprobe beweist keine Kollisionsfreiheit von SHA-256.
- NPC-Leak-Scan prüft konkrete Keys und Werte plus gezielte Nichtinterferenz. Kein allgemeiner Sicherheitsbeweis für beliebige spätere Dialog-/UI-Komposition.
- Performancewerte stammen von einem Serverprozess, nicht vom Handy. Dichte Graphen und alle-Personen-in-allen-Events sind absichtliche Stressmuster.
- Familienlabels sind keine zusätzliche Domain-Semantik. Tod, Eigentum, technische Zustandswechsel und Magieregeln wurden nicht durch Freitext als prüfbar ausgegeben.
- Es wurden keine kanonischen Änderungen vorgenommen. Alle neuen Dateien sind Bericht, Testinstrumente und Wegwerf-Fixtures.

E. Beispiel einer konkreten Reparatur

PRODUCT JUDGMENT

Ein Autor ändert complete auf partial und lässt eine nicht zugewiesene Person weiterhin als required:false stehen. Der Parser meldet requiredConclusions[i].value: resolves to undetermined. Fachlich erlaubte Reparaturen: die erforderliche Frage auf bekannte Aussagen begrenzen oder nach bewusster vollständiger Fallfestlegung complete verwenden. Unzulässige „Reparatur“: unbekannte Verantwortung ohne Begründung als falsch behaupten. Das Werkzeug sollte diese Entscheidung sichtbar machen, nicht selbst treffen.